DATA 266 Homework 1

Student ID: 018452122

1. Autoregressive Models

An autoregressive model predicts the next value in a sequence using previously observed values. A language model predicts the next token from previous tokens, while a time series model can predict future temperature, sales, demand, or stock prices from earlier observations. Real world examples include chatbots, predictive text, speech recognition, weather forecasting, financial forecasting, and product demand forecasting. These models are useful when the order of observations matters because predictions are generated step by step.

2. Diabetes Neural Network Experiment

The diabetes dataset was used for binary classification. The first eight columns were used as features and the final column was used as the binary target. The data was split into 70 percent training, 15 percent validation, and 15 percent testing data using a fixed random state. Standardization was fitted only on the training data and then applied to the validation and testing data.

Parameters and Configurations

SID4 = 2122
SEED = 2122
HP_ID = 4

The baseline model used hidden layers [64, 32], learning rate 0.001, and 30 epochs. The modified HP_ID 4 model used hidden layers [64, 32], learning rate 0.001, and 15 epochs. Thus, the modification was a shorter training schedule.

Accuracy Results

Framework	Model	2122	2123	2124	Mean	Std. dev.
PyTorch	Baseline	78.95%	76.32%	76.32%	78.36%	0.51%
PyTorch	Modified HP_ID 4	75.44%	76.32%	78.95%	76.90%	0.51%
TensorFlow	Baseline	77.19%	77.19%	78.07%	77.49%	0.51%
TensorFlow	Modified HP_ID 4	73.68%	73.68%	77.19%	74.85%	1.34%

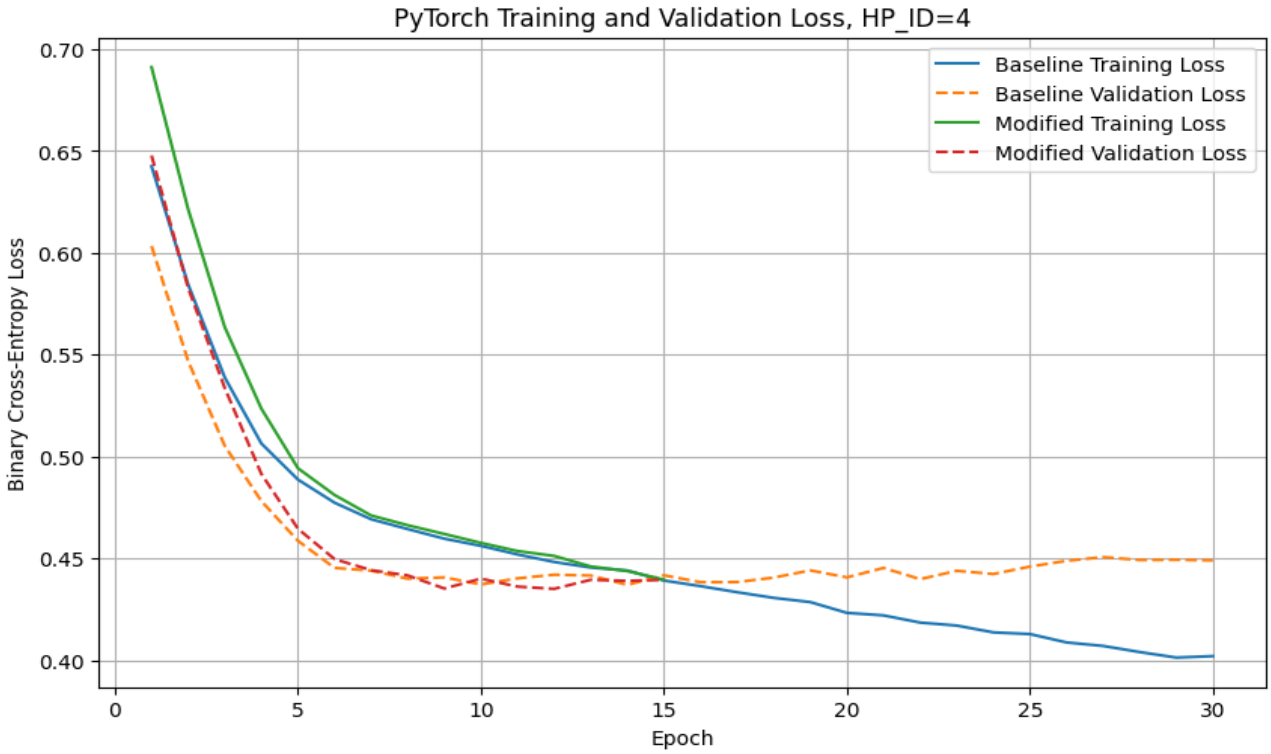
The baseline model achieved higher mean test accuracy than the modified model in both frameworks.

Loss Curve Analysis

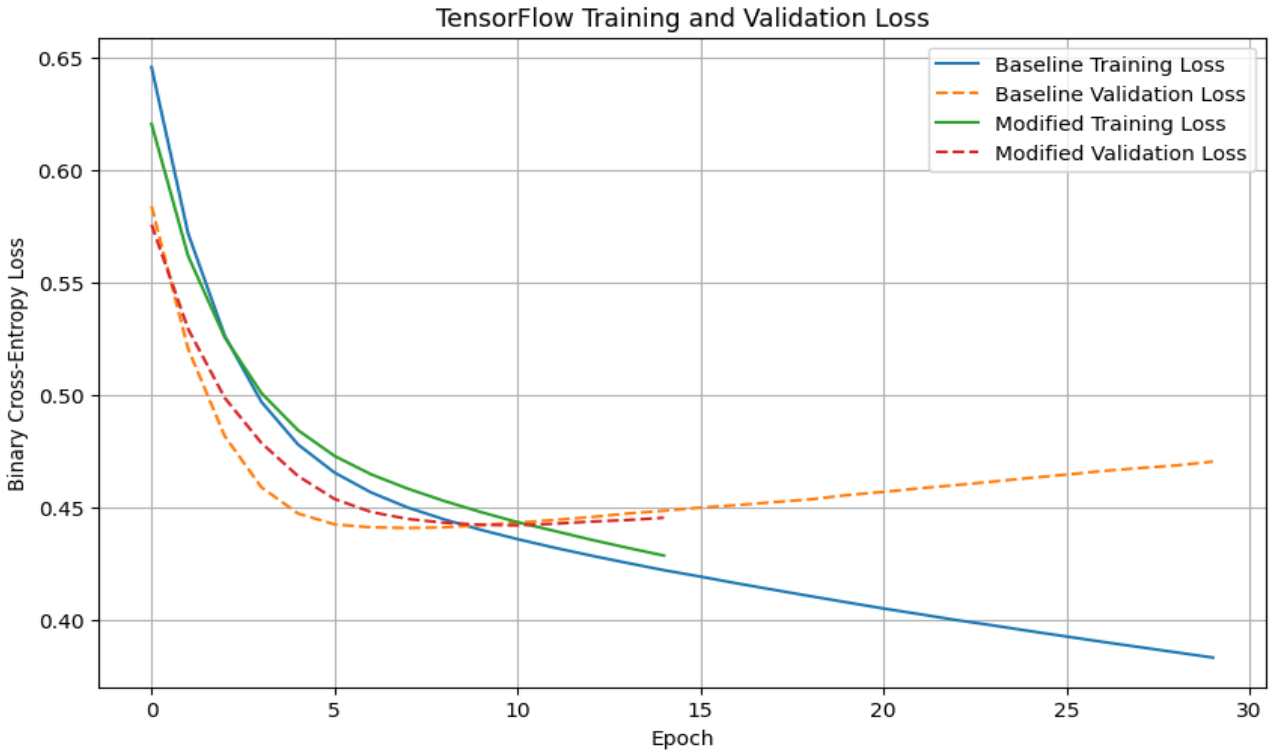
For PyTorch, the baseline reached its lowest validation loss of approximately 0.4370 at epoch 14. The modified model reached a slightly lower minimum of approximately 0.4351 at epoch 12, but its mean test accuracy was lower. Therefore, the modified PyTorch model was worse overall based on test accuracy. The modified curve did not show severe overfitting during its shorter 15 epoch run.

For TensorFlow, the baseline reached its lowest validation loss of approximately 0.4410 at epoch 8. The modified model reached approximately 0.4422 at epoch 11. The modified model also had lower mean test accuracy, so it was worse than the baseline. The shorter training schedule appears to have limited the amount of learning completed.

Training and Validation Loss Curves



Training and Validation Loss Curves



3. CUDA Matrix Multiplication

The CUDA implementation uses a two dimensional grid of blocks. Each block contains 16 by 16 threads, for 256 threads per block. Each thread calculates one element of the output matrix C. The row and column are determined from blockIdx, blockDim, and threadIdx. Boundary checks prevent invalid memory accesses.

The program was compiled with NVIDIA nvcc and profiled using NVIDIA Nsight Compute. The profiler command was ncu --set full --target-processes all ./cuda_benchmark. CUDA events measured kernel execution and the host to device plus device to host transfer interval separately.

Total CUDA notebook runtime: 0:35:34.985702

CUDA Timing Results

Matrix size	CPU (ms)	GPU kernel (ms)	H2D plus D2H (ms)	Speedup
256	20.107	21.066	0.293	0.941
1024	3274.540	9.168	3.095	267.034
4096	817454.761	418.377	47.463	1754.797

The smallest tested matrix size at which GPU end to end time became beneficial was 1024 by 1024. At 256 by 256, fixed kernel launch and data transfer overhead made the GPU slightly slower. At larger sizes, the GPU distributed many more operations across its threads and became substantially faster. The crossover is not at size zero because launching the kernel and moving data still require time even when the computation is small.